



TENEXA

Project Assignment PI Fullstack JS

Esprit Team Name: Eventify

11-02-2026

Contents

1	Project Introduction	3
1.1	Background and Motivation	3
2	Study of the Existing	4
3	Project Objective	5
4	Business Features Specifications	6
4.1	Functional Requirements	6
4.1.1	Authentication & Multi-tenancy	6
4.1.2	Configuration Module	6
4.1.3	Sales & Purchases Module	6
4.1.4	Inventory Management	6
4.1.5	AI & Insights Module	6
4.2	Non-Functional Requirements	7
5	Detailed User Stories (Product Backlog)	8
6	System Design and Architecture	10
6.1	Use Case Diagram	10
6.2	High-Level Architecture Diagram	11
7	Data Model	12
7.1	Key Business Objects	12
7.2	Conceptual Data Model (Class Diagram)	13
7.3	Conceptual Data Model (MCD)	14
7.4	Logical Data Model (MLD)	14
8	Appendix: Example AI Payloads (payload_json)	19
9	Technology Stack Choice	21
10	Security and Compliance Considerations	23

11 CI/CD and DevOps Strategy	24
12 Testing Strategy	25
13 Timeline and Sprint Planning	26

Chapter 1

Project Introduction

1.1 Background and Motivation

In the current economic landscape, Small and Medium Enterprises (SMEs) struggle not only with digitizing their data but with deriving actionable value from it. Many companies still rely on manual CSV exports or basic spreadsheets to manage Sales and Purchases. While they possess the raw data, they lack the tools to analyze trends, predict stock shortages, or objectively assess their financial health.

The motivation behind this project is to bridge the gap between raw data storage and intelligent decision-making. By automating data ingestion and applying Machine Learning, we transform static historical data into predictive insights.

Chapter 2

Study of the Existing

Currently, the market features solutions like Swiver (a popular Tunisian invoicing and management platform).

Existing (Swiver): Excellent for operational management (creating invoices, managing cash flow, tax declarations). However, it functions primarily as a record-keeping and process-management tool. It lacks advanced predictive capabilities.

Gap: Existing tools tell you what happened (Descriptive Analytics) but rarely tell you what will happen (Predictive Analytics) or why it matters (Diagnostic Analytics).

Our Solution: Unlike Swiver, our project focuses on Business Intelligence and AI. We do not just store invoices; we analyze uploaded history to predict stockouts, recommend reorder quantities, and score company health automatically.

Chapter 3

Project Objective

To develop a multi-tenant SaaS (Software as a Service) Platform that allows companies to:

- Centralize Data: Ingest Sales and Purchases history via CSV files.
- Visualize Performance: View real-time dashboards for revenue, expenses, and profit.
- Leverage AI: Use Machine Learning to predict stock run-outs, identify best/worst products, and generate an automated Company Health Score.

Chapter 4

Business Features Specifications

4.1 Functional Requirements

The system is divided into modular services providing the following key functionalities:

4.1.1 Authentication & Multi-tenancy

- Role-Based Access Control (Super Admin vs Company Admin, Accountant).
- Strict data isolation (Company A cannot see Company B's data).

4.1.2 Configuration Module

Management of company settings (Tax rate, Currency, Sector).

4.1.3 Sales & Purchases Module

- CSV Import/Upload mechanism with validation.
- Automated parsing and storage into structured databases.
- Calculation of Total Revenue and Total Expenses.

4.1.4 Inventory Management

Real-time stock tracking based on movements (Sales = OUT, Purchases = IN).

4.1.5 AI & Insights Module

- Sales Analysis: Classification of best/worst performing products.
- Inventory AI: Prediction of stockout dates and reorder quantity recommendations.

- Report AI: Generation of a global Health Score (0–100) with explainable insights.

4.2 Non-Functional Requirements

- Scalability: Horizontal scaling using microservices.
- Performance: Dashboards load within 2 seconds; CSV processing is asynchronous.
- Security:
 - JWT for stateless authentication.
 - Data isolation via `company_id`.
- Maintainability: Separation of concerns (Frontend, API Gateway, Microservices, ML Worker).

Chapter 5

Detailed User Stories (Product Backlog)

ID	Module	User Story	Role
PB-01	Auth	Secure login	Super Admin, Company Admin
PB-02	Auth	View all companies	Super Admin
PB-03	Auth	Access only own company data	Company Admin
PB-04	Config	Configure company info	Company Admin
PB-05	Config	View company configurations	Super Admin
PB-06	Sales	Upload sales CSV	Company Admin
PB-07	Sales	Visualize sales evolution	Company Admin
PB-08	Sales AI	Identify best/worst products	Company Admin
PB-09	Sales AI	View sales insights for all companies	Super Admin
PB-10	Purchases	Upload purchases CSV	Company Admin
PB-11	Purchases	Track total expenses	Company Admin
PB-12	Inventory	View current stock per item	Company Admin
PB-13	Inventory	Predict stock run-out	Company Admin
	AI		
PB-14	Inventory	Recommend reorder quantity	Company Admin
	AI		
PB-15	Inventory	Receive stock risk alerts	Company Admin
PB-16	Inventory	Monitor inventory risks across companies	Super Admin
PB-17	ML Engine	Trigger ML automatically after upload	System
PB-18	ML Engine	Prepare clean datasets for ML	System
PB-19	ML Engine	Store ML outputs for dashboards	System

PB-20	Report	View revenue, costs, profit, and taxes	Company Admin
PB-21	Report AI	Generate company health score	Company Admin
PB-22	Report AI	View AI health score explanation	Company Admin
PB-23	Report	View reports of all companies	Super Admin
PB-24	UX	Role-based dashboards	All Users
PB-25	UX	Auto-update dashboards after uploads	Company Admin

Chapter 6

System Design and Architecture

6.1 Use Case Diagram



Figure 6.1: Use Case Diagram of the TENEXA Platform

6.2 High-Level Architecture Diagram

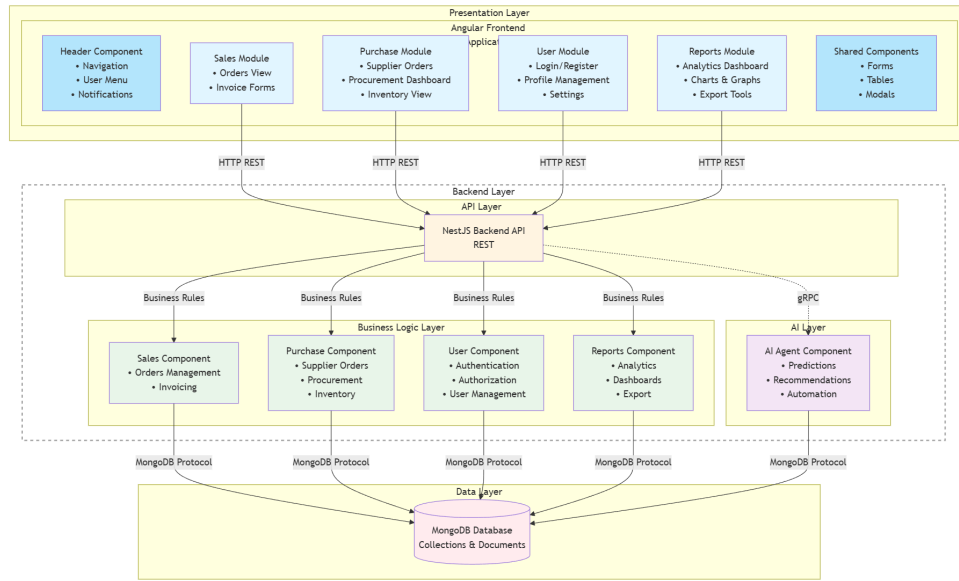


Figure 6.2: High-Level Architecture of the TENEXA Platform

Chapter 7

Data Model

7.1 Key Business Objects

Element	Description
Company	Tenant boundary. All data is scoped by company.
Product & Category	Catalog used by Sales and Purchases.
Client & Supplier	Business partners involved in Orders and Purchases.
Order (Commande)	Sales document created from uploaded sales rows.
Purchase (Achat)	Purchases/expenses document created from uploaded purchase rows.
Stock	Computed from movements (IN / OUT / ADJUST) for traceability.
AI Insights	Machine Learning outputs stored for dashboards (sales, purchases, report).

Table 7.1: Key Business Objects Description

7.2 Conceptual Data Model (Class Diagram)

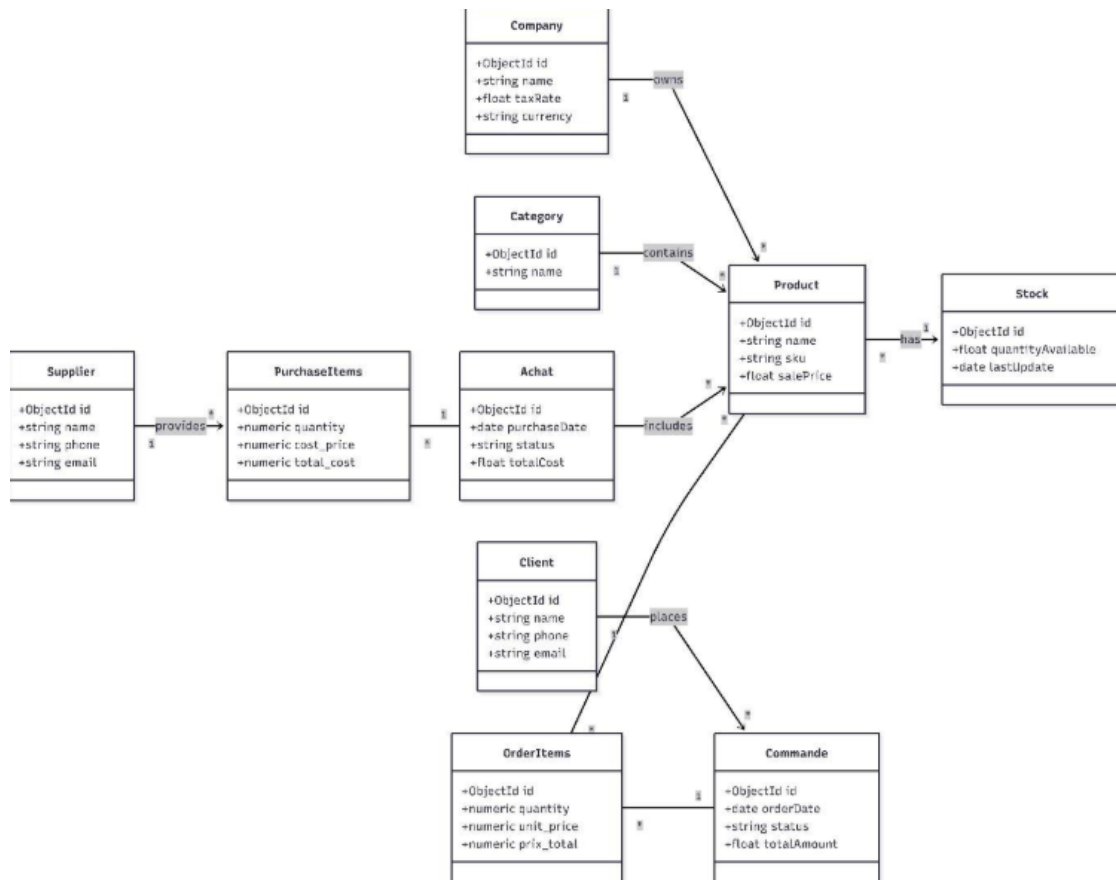


Figure 7.1: Conceptual Class Diagram of the TENEXA System

7.3 Conceptual Data Model (MCD)

Entities and high-level relationships (cardinalities):

MCD – Relationships

From	Cardinality	To
Company	1–N	Products
Category	1–N	Products
Client	1–N	Orders
Order	1–N	OrderItems
Product	1–N	OrderItems
Supplier	1–N	Purchases
Purchase	1–N	PurchaseItems
Product	1–N	PurchaseItems
Product	1–N	StockMovements
Company	1–N	AllInsights

Table 7.2: Conceptual Data Model (MCD) – Entity Relationships

7.4 Logical Data Model (MLD)

Normalized tables with primary keys (PK) and foreign keys (FK).

MLD – Tables Overview

Table	Primary Key	Key Attributes (summary)
users	user_id	email, password_hash, role, fkcompany_id
companies	company_id	name, tax_rate, currency, address, matricule_fiscal, code_postal, secteur_d'activité, mail
categories	category_id	company_id (FK), name
products	product_id	company_id (FK), category_id (FK), name, sale_price
suppliers	supplier_id	company_id (FK), name, phone, email
clients	client_id	company_id (FK), name, phone, email
orders	order_id	company_id (FK), client_id (FK), order_date, status, total_amount
order_items	order_item_id	order_id (FK), product_id (FK), quantity, unit_price, prix_total
purchases	purchase_id	company_id (FK), supplier_id (FK), purchase_date, status, total_cost
purchase_items	purchase_item_id	purchase_id (FK), product_id (FK), quantity, cost_price, total_cost
stock_movements	movement_id	company_id (FK), product_id (FK), type, quantity, ref_type

Table 7.3: Logical Data Model (MLD) – Tables Overview

3. Physical Data Model (MPD)

Companies

Column	Type	Constraints / Notes
company_id	long	PK
name	VARCHAR(200)	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()
tax_rate	NUMERIC(5,4)	NOT NULL, default 0.1900
currency	VARCHAR(10)	NOT NULL, default 'TND'
updated_at	TIMESTAMP	DEFAULT NOW()

Table 7.4: Physical Data Model for the Companies Collection

Products

Product catalog used by Sales and Purchases.

Column	Type	Constraints / Notes
product_id	long	PK
company_id	BIGINT	FK → companies, NOT NULL
category_id	BIGINT	FK → categories, nullable
name	VARCHAR(200)	NOT NULL
sale_price	NUMERIC(12,3)	≥ 0, default 0
is_active	BOOLEAN	default TRUE
created_at	TIMESTAMP	DEFAULT NOW()

Table 7.5: Physical Data Model for the Products Collection

Orders

Sales documents created from Sales CSV rows.

Column	Type	Constraints / Notes
orders.order_id	BIGSERIAL	PK
orders.company_id	BIGINT	FK → companies, NOT NULL
orders.client_id	BIGINT	FK → clients, nullable
orders.order_date	DATE	NOT NULL
orders.total_amount	NUMERIC(14,3)	≥ 0, default 0

Table 7.6: Physical Data Model for the Orders Table

Order Items

Stores per-product lines of Sales documents.

Column	Type	Constraints / Notes
order_items.order_item_id	BIGSERIAL	PK
order_items.order_id	BIGINT	FK → orders, ON DELETE CASCADE
order_items.product_id	BIGINT	FK → products
order_items.quantity	NUMERIC(12,3)	> 0
order_items.unit_price	NUMERIC(12,3)	≥ 0
order_items.line_total	NUMERIC(14,3)	≥ 0

Table 7.7: Physical Data Model for the Order Items Table

Purchases

Purchases/expenses documents created from Purchases CSV rows.

Column	Type	Constraints / Notes
purchases.purchase_id	BIGSERIAL	PK
purchases.company_id	BIGINT	FK → companies, NOT NULL
purchases.supplier_id	BIGINT	FK → suppliers, nullable
purchases.purchase_date	DATE	NOT NULL
purchases.total_cost	NUMERIC(14,3)	≥ 0, default 0

Table 7.8: Physical Data Model for the Purchases Table

Purchase Items

Stores per-product cost lines used in the final report.

Column	Type	Constraints / Notes
purchase_items.purchase_item_id	BIGSERIAL	PK
purchase_items.purchase_id	BIGINT	FK → purchases, ON DELETE CASCADE
purchase_items.product_id	BIGINT	FK → products
purchase_items.quantity	NUMERIC(12,3)	> 0
purchase_items.cost_price	NUMERIC(12,3)	≥ 0
purchase_items.line_total_cost	NUMERIC(14,3)	≥ 0 (quantity * cost_price)

Table 7.9: Physical Data Model for the Purchase Items Table

Stock Movements

Traceable stock model used for inventory and ML stockout prediction (IN=from purchases, OUT=from sales).

Column	Type	Constraints / Notes
movement_id	BIGSERIAL	PK
company_id	BIGINT	FK → companies, NOT NULL
product_id	BIGINT	FK → products, NOT NULL
movement_type	VARCHAR(10)	IN OUT ADJUST
quantity	NUMERIC(12,3)	> 0
movement_date	TIMESTAMP	DEFAULT NOW()
ref_type	VARCHAR(20)	ORDER PURCHASE ADJUST

Table 7.10: Physical Data Model for the Stock Movements Table

AI Insights

Stores ML outputs for dashboards (Sales: best/worst product, Purchases: stock-out/reorder, Report: health score).

Column	Type	Constraints / Notes
insight_id	BIGSERIAL	PK
company_id	BIGINT	FK → companies, NOT NULL
module	VARCHAR(20)	sales purchases report
generated_at	TIMESTAMP	DEFAULT NOW()
payload_json	JSONB	Model outputs stored as JSON

Table 7.11: Physical Data Model for the AI Insights Table

Chapter 8

Appendix: Example AI Payloads (payload_json)

These examples illustrate the JSON stored in the `payload_json` column of the `ai_insights` table for different modules.

Sales Module

```
{
  "best_product": "Laptop HP",
  "best_revenue": 28000,
  "worst_product": "Keyboard",
  "worst_revenue": 900
}
```

Purchases Module

```
{
  "item": "Laptop HP",
  "predicted_stockout_days": 3,
  "recommended_reorder": 18,
  "risk_level": "High"
}
```

Report Module

```
{
  "health_score": 82,
}
```

```
"status": "Healthy",  
"main_reason": "Strong margin + stable sales"  
}
```

Chapter 9

Technology Stack Choice

The TENEXA platform uses a multi-layered architecture with the following technology choices:

Layer	Technology	Reason for Choice
Frontend	Angular	Strong typing (TypeScript), component-based structure, excellent for enterprise dashboards.
Backend	NestJS	Modular architecture (Modules/Controllers/Services), native support for Microservices, TypeScript.
ML Engine	Python	Unrivaled libraries for Data Science (Pandas, Scikit-Learn).
Database	MongoDB	Flexible schema is ideal for storing variable CSV data and JSON AI payloads.
Containerization	Docker	Ensures environment consistency across Development, Testing, and Production.

Table 9.1: Technology Stack Choice for the TENEXA Platform

Additional Layers:

- **Entry Point:** NestJS , handling routing and authentication (Auth Guard).
- **Services Layer:** Distinct NestJS distributed application for business logic:
 - user Service

- Sales and Purchases Service
 - Inventory Service
 - agent ia service
- **Intelligence Layer:** Python ML Service (FastAPI/Worker), which receives triggers from Node.js, processes data using Pandas/Scikit-Learn, and saves results.
- **Data Layer:** MongoDB Cluster, with data logically segregated by `company_id`.

Chapter 10

Security and Compliance Considerations

The TENEXA platform implements several security measures to ensure data privacy, integrity, and tenant isolation.

- **Authentication:** Implemented via Passport.js and JWT. Tokens expire after a set time to reduce risks from token theft.
- **Authorization:** Role-Based Access Control (RBAC) ensures that "Company Admins" cannot access "Super Admin" routes.
- **Tenant Isolation:** A custom Interceptor/Middleware automatically attaches `company_id` from the JWT to every database query to prevent data leaks between companies.
- **Input Validation:** `class-validator` and DTOs in NestJS prevent injection attacks and ensure CSV data integrity.

Chapter 11

CI/CD and DevOps Strategy

- **Version Control Plan:**
 - **Strategy:** Git Flow
 - **Branches:**
 - * **main:** Production-ready code
 - * **develop:** Integration/testing branch
 - * **feature/xyz:** Branches for specific user stories (e.g., CSV upload)
 - Repository hosted on GitHub with branch protection rules.
- **CI/CD Pipeline Setup:** Implemented with GitHub Actions.
 - **Trigger:** On push/pull request to **develop** or **main**
 - **Install:** Setup Node.js and Python environments; install dependencies (`npm install`, `pip install`)
 - **Lint:** ESLint (TS) and Flake8 (Python) for code quality
 - **Test:** Unit tests (Jest for NestJS, PyTest for Python)
 - **Build:** Build Angular app and NestJS app to ensure no compilation errors
 - **Docker Validation:** Verify that `docker build` commands succeed for all services

Chapter 12

Testing Strategy

- **Unit Tests:** Focus on individual service methods (e.g., calculating Total Revenue)
- **Integration Tests:** Verify communication between front-end and backend
- **End-to-End (E2E) Tests:** Verify full flow from CSV upload to Dashboard visualization

Chapter 13

Timeline and Sprint Planning

- **Sprint 1: User model 1 week**
 - **Focus:** Project setup (Repo, Docker), Authentication Service (Login/Register), Company Configuration
 - **Goal:** A user can log in and configure their company profile
- **Sprint 2:IA Agent Model 2 weeks**
 - **Focus:** AI-powered predictions, personalized recommendations, product segmentation, and streamlined user experience
 - **Goal:** To use AI to help users make smarter decisions, save time, personalize interactions, and improve business outcomes
- **Sprint 3: Sales and Purchases Model 1 week**
 - **Focus:** Dashboard creation, aggregating data for charts (Revenue over time, Expenses vs. Income), managing orders and invoices
 - **Goal:** Visual representation of uploaded data for better decision-making
- **Sprint 4: Inventory Management 2 weeks**
 - **Focus:** Real-time stock tracking based on movements
 - **Goal:** To maintain optimal inventory levels by tracking every product movement in real-time, ensuring you never run out of critical items while avoiding excess stock that wastes money.